

(3rd year) Week 1 Module

→ Star pattern - (using C++)

- Square

```
*****
*****
*****
*****
*****
```

- Logic:-
1. Take input n .
 2. Use the outer loop to control rows.
 3. Use the inner loop to print n stars in each row.
 4. After printing one row, move to the next line.
 5. Repeat the process for all rows.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of X: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << "*";
        }
        cout << "\n";
    }
    return 0;
}
```

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?)  
{ g++ module1_P1_sqaure.cpp -o module1_P1_sqaure } ; if ($?) { .\module1_P1_sqaure }  
enter the value of X: 5  
*****  
*****  
*****  
*****  
*****
```

• Hollow Square

```
* * * * *
*       *
*       *
*       *
* * * * *
```

- Logic:-
1. Take input 'n'.
 2. Use two nested loops for rows and columns.
 3. Outer for rows and inner for columns.
 4. Check whether the current element lies on the boundary.
 5. If it is the first row, last row, first column or last column, print "*".
 6. Otherwise print space.
 7. Move to the next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter a range range ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {
                cout << "*";
            }
            else {
                cout << " ";
            }
        }
        cout << "\n";
    }
}
```

Code - star pattern + [icon] [icon] ... [icon] [icon]

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?) { g++ module1_P2_hollow_sqr.cpp -o module1_P2_hollow_sqr } ; if ($?) { .\module1_P2_hollow_sqr }
enter a range : 5
*****
*   *
*   *
*   *
*   *
*****
```

• Triangle

```
*  
**  
***  
****  
*****
```

Logic:- 1. take input n

2. Run outer loop with condition $i < x$.

3. Run inner loop $j \leq i+1$ to print " $*$ ";

4. print next line after each row.

Code:->

```
#include <iostream>.  
using namespace std;  
int main() {  
    int x;  
    cout << "Enter the value of x";  
    cin >> x;  
    for (int i = 0; i < x; i++) {  
        for (int j = 1; j <= i+1; j++) {  
            cout << "*";  
        }  
        cout << "\n";  
    }  
    cout << endl;  
}
```


• Inverted Triangle Pattern

```
* * * * *  
* * * *  
* * *  
* *  
*
```

Logic:- 1. Take input x .

2. Run outer loop " $i < x$ ".

3. Run inner loop in descending order to print star.

4. Print next line after each row.

Code:-

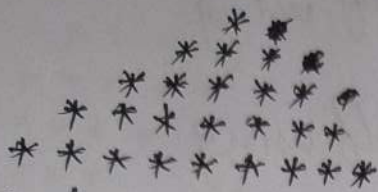
```
#include <iostream>  
using namespace std;  
int main() {  
    int x;  
    cout << "enter the value of x";  
    cin >> x;  
    for (int i = 0; i < x; i++) {  
        for (int j = x; j >= i + 1; j--) {  
            cout << "*";  
        }  
        cout << "\n";  
    }  
    cout << endl;  
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Code - star pattern + - [] [] ... | [] [] X

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?) { g++ module1_P4_Inverted_Triangle.cpp -o module1_P4_Inverted_Triangle } ; if ($?) { .\module1_P4_Inverted_Triangle }
enter the value of x 5
*****
****
***
**
*
```

• Pyramid



Code:-

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "enter the number" ;
```

```
    cin >> n;
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < (n - i - 1); j++) {
```

```
            cout << " ";
```

```
        }
```

```
        for (int k = 0; k < (2 * i + 1); k++) {
```

```
            cout << "*";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
}
```

Logic:- 1. Take input n.

2. print spaces using $j < n - i - 1$.

3. print stars using $k < 2 * i + 1$.

4. print next line after each row.

Code - star pattern + ▾ □ □ ... □ ×

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?) { g++ module1_P5_Pyramid.cpp -o module1_P5_Pyramid }
amid } ; if ($?) { .\module1_P5_Pyramid }
enter the number: 5
*
***
*****
*****
*****
*****
```

• Diamond:-

```
  *
 * * *
* * * * *
* * * * *
 * * *
  *
```

Logic:- 1. take input n.

2. print upper half using
 $K < 2*i + 1$;

3. print lower half using
 $K < 2*(n-i) - 1$.

4. Use spaces for alignment.

5. print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of n: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = n - 1; j > i; j++) {
            cout << " ";
        }
        for (int k = 0; k < (2 * i + 1); k++) {
            cout << "* ";
        }
        cout << "\n";
    }
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            cout << " ";
        }
        for (int k = 0; k < (2 * (n - i) - 1); k++) {
            cout << " ";
        }
        cout << "\n";
    }
    return 0;
}
```


• Half Diamond

```
*
**
***
****
*****
****
***
**
*
```

Logic:- 1. Take input n
2. print upper half using
 $j \leq i+1$

3. Number of stars first increase and then decreases.

4. Print lower half using decreasing stars.

5. Print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cout << "enter the value of x ";
    cin >> x;
    for (int i = 0; i < x; i++) {
        for (int j = 1; j <= i+1; j++) {
            cout << "*";
        }
        cout << "\n";
    }
    for (int i = 1; i < x; i++) {
        for (int j = x; j > i; j--) {
            cout << "*";
        }
        cout << endl;
    }
}
```

Code - star pattern + - [] ... [] x

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?) { g++ module1_P7_Half_Diamond.cpp -o module1_P7_Half_Diamond } ; if ($?) { .\module1_P7_Half_Diamond }
enter the value of x 5
*
**
***
****
*****
*****
****
***
**
*
```

• Inverted Half Diamond :-

```
* * * * *
 * * * *
  * * *
   * *
    *
```

Logic → 1) Take input n.
2) Print spaces first
3) Print stars in decreasing order.
4) Print next line after each row.

Code :-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of n: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            cout << " ";
        }
        for (int k = 0; k < n - i; k++) {
            cout << "*" << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - star pattern + - [ ] [ ] ... | [ ] [ ] X
```

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\star pattern\" ; if ($?) { g++ module1_P8_Half_Diamond_inverted.cpp -o module1_P8_Half_Diamond_inverted } ; if ($?) { .\module1_P8_Half_Diamond_inverted }
enter the value of n: 5
* * * * *
* * * *
* * *
* *
*
```

→ Number Pattern

• Square

- Logic :-
- 1) take input n
 - 2) Run outer loop $i < n$
 - 3) Run inner loop $j < n$
 - 4) Print next line after each row.

Code :-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter the number: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << 1;
        }
        cout << "\n";
    }
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - number pattern + - [ ] [ ] ... | [ ] [ ] X
```

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_1_sqr.cpp -o module_1_np_1
_sqr } ; if ($?) { .\module_1_np_1_sqr }
enter the number : 5
11111
11111
11111
11111
11111
```

• Incrementing square pattern

Logic :- 1) Take input n

- 2) Run outer loop $i < n$.
- 3) Run inner loop ~~to~~ to print from 1 to n
- 4) Print next line after each row.

Code :-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the rang: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << j;
        }
        cout << "\n";
    }
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - number pattern + - [ ] [ ] ... [ ] [ ] X
```

```
● PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_2_Incrementing_sqr.cpp -o module_1_np_2_Incrementing_sqr } ; if ($?) { .\module_1_np_2_Incrementing_sqr }
enter the range : 5
12345
12345
12345
12345
12345
```

• Number triangle pattern.

Logic → Take input n

- 1) Run outer loop $i < n$.
- 2) Run inner loop $j \leq i+1$.
- 3) Print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the range: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 1; j <= i+1; j++) {
            cout << j;
        }
        cout << endl;
    }
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - number pattern + - [ ] ... | [ ] X
```

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_3_Right_Triangle.cpp -o module_1_np_3_Right_Triangle } ; if ($?) { .\module_1_np_3_Right_Triangle }
enter the range : 5
1
12
123
1234
12345
```

• Inverted Number pattern.

- Logic:-
- 1) take n as input.
 - 2) Print spaces first.
 - 3) Print numbers in decreasing order.
 - 4) Print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of n: ";
    (cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            cout << " ";
        }
        for (int k = 0; k < n - i; k++) {
            cout << i + 3 << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

Code - number pattern + - ... | [] ×

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_4_Inverted_Diamond.cpp -o module_1_np_4_Inverted_Diamond } ; if ($?) { .\module_1_np_4_Inverted_Diamond }
enter the value of n: 5
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

• Sandwich pattern.

- Logic:-
- 1) Print upper half using loops.
 - 2) Print lower half in reverse order.
 - 3) use spaces for alignment
 - 4) Print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of n: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            cout << " ";
        }
        for (int k = 0; k < (2 * (n - i) - 1); k++) {
            cout << n + i;
        }
        cout << "\n";
    }
    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j < i; j++) {
            cout << " ";
        }
        for (int k = 0; k < (2 * (n - i) - 1); k++) {
            cout << n + i;
        }
        cout << "\n";
    }
    return 0;
}
```

Code - number pattern + - [] [] ... [] []

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_5_Sandwich_Pattern.cpp -o module_1_np_5_Sandwich_Pattern } ; if ($?) { .\module_1_np_5_Sandwich_Pattern }
enter the value of n: 5
555555555
  6666666
    77777
      888
        9
      888
    77777
  6666666
555555555
```

~~11/6~~

• Incrementing Diamond pattern

- Logic →
- Print upper half using $J \leq i+1$.
 - Print lower half in reverse order.
 - Numbers first increase and then decrease.
 - Print next line after each row.

Code:-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "enter the value of x ";
    cin >> x;
    for (int i = 0; i < x; i++) {
        for (int j = 1; j <= i+1; j++) {
            cout << x+i;
        }
        cout << "\n";
    }
    for (int i = 1; i < x; i++) {
        for (int j = x; j > i; j--) {
            cout << x+i;
        }
        cout << endl;
    }
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - number pattern + v [ ] [X] ... | [ ] [X] X
```

```
PS D:\codez\DS CPP module1> cd "d:\codez\DS CPP module1\number pattern\" ; if ($?) { g++ module_1_np_6_Incrementing_Diamond_Pattern.cpp -o module_1_np_6_Incrementing_Diamond_Pattern } ; if ($?) { .\module_1_np_6_Incrementing_Diamond_Pattern }  
enter the value of x 5  
5  
66  
777  
8888  
99999  
6666  
777  
88  
9
```